Robotic Milling with a Mitsubishi RV-8CRL Industrial Arm: Toolpath Planning, TCP Calibration, and Chatter Compensation

Aayush Shah, Ishaan Monda, Shashwat Agrawal, Kartik Goyal
ME230 Project Report
Department of Mechanical Engineering

Abstract

This report documents the end-to-end process of performing robotic milling using a Mitsubishi RV-8CRL industrial robot arm in combination with RoboDK for toolpath simulation and validation. The project involved designing a target geometry in Autodesk Fusion 360, generating machining toolpaths, calibrating the tool center point (TCP) using a frame-based method, and executing slot milling on a physical workpiece. A key finding was the presence of an upward ramp in the milled slot caused by structural compliance in the robot arm. An initial compensation strategy was developed by introducing a deliberate downward plunge in the toolpath, reducing the depth variation from approximately 200 micrometers to 85 micrometers. The report also discusses plans for curved surface milling and outlines a number of directions for future improvement. A companion video tutorial is attached alongside this report.

1 Introduction

Industrial robotic arms are increasingly being explored as flexible alternatives to traditional CNC machining centers. Unlike a dedicated milling machine, a robot arm can reach complex orientations and adapt to a variety of workpiece geometries without needing specialized fixturing. At the same time, robots come with their own set of challenges: most notably, they are significantly less stiff than machine tools, which makes them susceptible to deflection and chatter during material removal.

This project used a Mitsubishi RV-8CRL six-axis robot arm to perform slot milling on a flat workpiece. The goal was to execute the complete workflow from CAD design to physical cutting, and in doing so, understand the practical constraints of using an articulated arm for precision material removal. The entire process was simulated and validated in RoboDK before any physical cutting was attempted, ensuring that the toolpath was safe and reachable within the robot's workspace.

A key insight from this project is that once this workflow is established, it can in principle be used to machine a part of virtually any geometry. Fusion 360 handles the geometry and toolpath planning, RoboDK handles the robot-specific kinematics, and the controller executes the result. The combination removes the geometric limitations of conventional 3-axis CNC and opens the door to complex curved surface machining using the same pipeline.

A secondary objective was to study the effect of robot compliance on the geometry of the milled slot, and to develop an initial compensation strategy to improve dimensional accuracy. The results of this experiment are presented in Section **??**.

2 Equipment and Software

2.1 Hardware

The primary hardware used in this project was the Mitsubishi RV-8CRL six-axis industrial robot arm, controlled by the CR800D controller. The robot was connected to the host computer over Ethernet using a TCP/IP connection. The robot's IP address was fixed at `192.168.0.20`, and the host machine was assigned any address in the `192.168.0.xx` subnet with a subnet mask of `255.255.255.0`. Communication took place over port `10001`, as specified by the CR800D controller.

A 3 mm flat-end milling cutter was mounted on the robot flange for all cutting experiments. Slot depth measurements after machining were taken us-

ing a laser displacement sensor, which provided non-contact, high-resolution readings of the slot profile along its length.

2.2 Software

Three software packages formed the core of the workflow.

Autodesk Fusion 360 was used for CAD modeling of the target geometry and for generating the machining toolpath using its built-in CAM environment. Fusion's CAM module produces toolpaths from a CNC machining perspective, outputting a dense sequence of end-effector positions in Cartesian space, with each position representing a setpoint for the tool tip.

RoboDK served as the critical bridge between the Fusion toolpath and the physical robot. Because Fusion's post-processors are oriented toward CNC machines rather than articulated arms, RoboDK takes the Cartesian end-effector positions produced by Fusion and solves the inverse kinematics to determine the joint angles required to reach each point. This is non-trivial for a six-axis arm, where multiple joint configurations can achieve the same end-effector position. RoboDK allows the operator to select and optimize over those configurations to avoid singularities, joint limit violations, and erratic transitions between setpoints. In addition to kinematics solving, RoboDK was used for simulation, frame definition, TCP setup, and export of the final robot program. A RoboDK license was required for this work, which was obtained on a six-month trial after reaching out directly to the RoboDK team.

RT ToolBox3 (Mitsubishi's proprietary programming environment) was used to load and execute the generated program on the robot controller. While RoboDK itself can also upload programs to the robot over a network connection, RT ToolBox3 was preferred for actual execution. Being native to the CR800D controller, it offers finer control over program execution, including real-time override speed adjustment, step-by-step program stepping, and live monitoring of which line is currently being executed. In addition, RT ToolBox3 allows the full program to be simulated in a controller-native environment as a second layer of validation, separate from the RoboDK simulation, giving further confidence that the program will behave correctly on the physical machine. For all of these reasons, RT ToolBox3 was used as the primary interface for running programs on the robot.

3 CAD Modeling and Toolpath Generation in Fusion 360

3.1 Part Design

The target geometry, a simple flat-bottomed slot, was modeled in Fusion 360. The slot was designed to be 1 mm deep with a uniform floor. The stock material was defined to match the physical workpiece dimensions, ensuring that the toolpath would account for the correct depth of cut and approach direction.

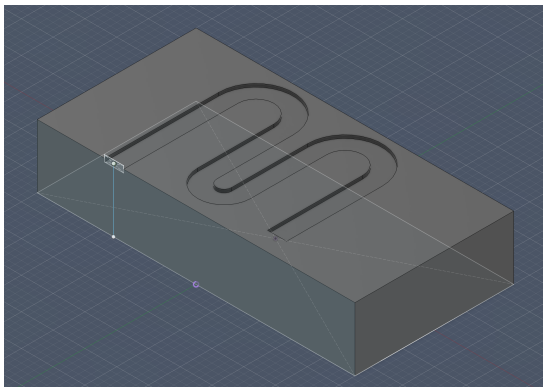

Figure 1: Target slot geometry modeled in Fusion 360.

3.2 CAM Setup and Operation Selection

Once the model was ready, a new CAM setup was created in Fusion's Manufacturing workspace. The coordinate system of the setup was aligned with the physical workpiece orientation so that the exported toolpath would be consistent with the robot's frame.

Two types of milling operations were considered during the project:

1. **Slotting**, where the tool cuts a straight channel along a defined path.
2. **Multi-axis operations**, which allow the tool to approach the workpiece from varying angles, suitable for more complex geometries.

For the primary slot milling experiments, slotting was used. Within the CAM environment, several options were explored for how the tool enters the material: the entry move can be configured as a straight plunge directly into the cut depth, a helical ramp that spirals down gradually, or a linear ramp along the slot direction. Each strategy has different implications for cutting forces and surface quality. The linking moves between passes, the number of depth passes, and the radial step-over between adjacent passes were also configured carefully. Getting

these parameters right is important both for surface finish and for the structural loads the robot must sustain during cutting.

3.3 Tool Selection and Parameters

The milling tool was selected from the Fusion tool library to match the physical 3 mm flat-end mill attached to the robot. Key parameters set in the CAM environment included spindle speed, feed rate, depth of cut, and step-over. All parameters were tuned to values appropriate for the robotic milling (of the material we wanted to machine which was some form of plastic), which generally operates at lower feed rates than conventional CNC machining due to the reduced stiffness of the arm.

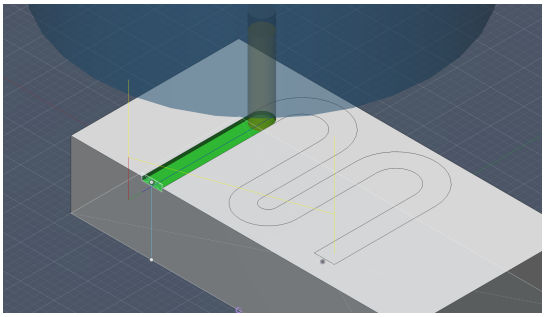

Figure 2: Toolpath preview in Fusion 360 CAM environment.

Once satisfied with the parameters, the toolpath was generated and verified visually within Fusion. The toolpath was then exported using the RoboDK post-processor plugin, which converts Fusion's intermediate toolpath format into a file compatible with RoboDK.

4 RoboDK Simulation and Validation

4.1 Importing the Model and Toolpath

The RoboDK plugin for Fusion 360 was used to directly export the toolpath into the RoboDK environment. The Mitsubishi RV-8CRL robot model was imported into RoboDK from its built-in library. The robot model mirrors the actual kinematic structure of the physical arm, including joint limits and reach envelope, making the simulation a reliable representation of what the real robot will do.

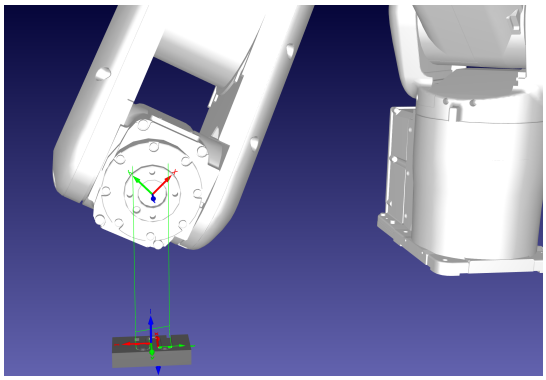

Figure 3: RoboDK simulation environment with imported robot and toolpath.

4.2 Frame Definition and Tool Setup

Before the simulation could be run meaningfully, the tool frame and the workpiece frame needed to be defined correctly. This is one of the more involved steps of the workflow and is described in detail in Section 5.

The imported program from Fusion was selected in RoboDK, and the appropriate robot, tool frame, and reference frame were assigned to it. The start position of the robot was set to a known home configuration, and the toolpath was generated within RoboDK.

4.3 Simulation and Verification

The toolpath was simulated in RoboDK before any physical execution. During simulation, close attention was paid to any erratic or out-of-range movements flagged by the interface. These indicate configurations where the robot approaches a singularity or exceeds a joint limit, and they must be resolved before running the program on the physical machine.

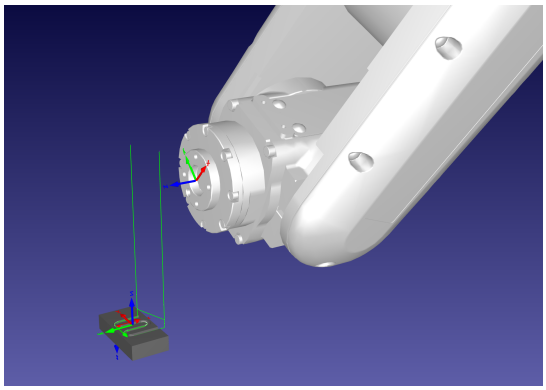

Figure 4: Toolpath simulation in RoboDK prior to physical execution.

Once the simulation looked clean, the program was exported as a `.prg` file. As an additional safety step, the program was also loaded into RT ToolBox3 and simulated there in offline mode, providing a second, controller-native validation of the motion before physical execution was attempted. Since RT ToolBox3 is native to the CR800D, this simulation gives a closer representation of how the controller will actually interpret and execute each instruction and is hence good from a safety stand-point.

5 TCP Calibration Using the Frame-Based Method

Accurate tool center point (TCP) calibration is critical for any robotic milling task. If the TCP is not defined correctly, there will be a systematic offset between the programmed path and where the tool actually cuts. In this project, TCP calibration was carried out using a sequence of manually defined reference frames.

5.1 Step 1: Defining a Flange Reference Frame

The robot's flange, the mounting surface for the tool, is oriented at a specific angle relative to the robot's base frame depending on the arm's current configuration. Because the flange's native orientation is not generally aligned with the world coordinate axes, direct caliper measurements from the flange face are difficult to interpret without first establishing a more convenient reference.

A new frame was defined at the robot flange with its orientation aligned to the robot's base frame axes. The origin of this frame was set at the flange center, using the flange pose read from the robot controller over the TCP/IP connection.

5.2 Step 2: Manual TCP Measurement

With the flange reference frame established, the physical offset from the flange face to the tool tip was measured manually using calipers. This included the axial protrusion of the cutter along the tool axis and any lateral offset arising from the tool holder geometry. The result of this step was the position of the TCP expressed in the coordinate system of the flange-aligned frame.

5.3 Step 3: Creating a Frame at the TCP

A second frame was created in RoboDK at the location of the TCP, placed at the measured offset values relative to the flange frame. This made the TCP's location explicit within the simulation environment and allowed it to be verified visually before committing it to the tool definition.

5.4 Step 4: Moving the Frame to Absolute Coordinates

The TCP frame was repositioned to the absolute coordinates of the tool tip relative to the flange-aligned frame. This transformation confirms that the TCP frame sits at the physically correct location in 3D space as represented in RoboDK.

5.5 Step 5: Finalizing the Tool Definition

Once the TCP frame was confirmed to be in the correct position, both intermediate frames were removed from the RoboDK model. The tool definition in RoboDK was updated with the measured TCP offset so that all subsequent motion commands correctly account for the actual geometry of the tool.

It is worth noting that any residual error in the manual caliper measurements or in the frame alignment carries through as a systematic error in all machining operations. Part of the depth variation observed in the milled slot, beyond what is attributable to arm compliance, may also reflect a small TCP calibration offset that shifts the effective cut depth slightly from the programmed value. This highlights the importance of a careful and repeatable calibration procedure, and motivates future work on automated TCP identification methods.

6 Program Transfer and Execution

6.1 Editing the Generated Program

The `.prg` file exported from RoboDK was opened in RT ToolBox3 in offline mode. Before loading it onto the controller, some header information automatically inserted by RoboDK, such as communication port opening commands and redundant preamble lines, was removed. After editing, the syntax of the file was verified to ensure it would be accepted by the controller without errors.

6.2 Loading and Running on the Controller

RT ToolBox3 was switched to online mode to establish a live connection with the Mitsubishi CR800D controller. From the programs folder in RT ToolBox3's offline section, the edited program was right-clicked and sent to the controller. Once transferred, the robot's operation panel was used to select the

program and begin the cutting process. The override speed was set to a conservative value for the first run, allowing us to monitor the motion and intervene if anything looked unexpected.

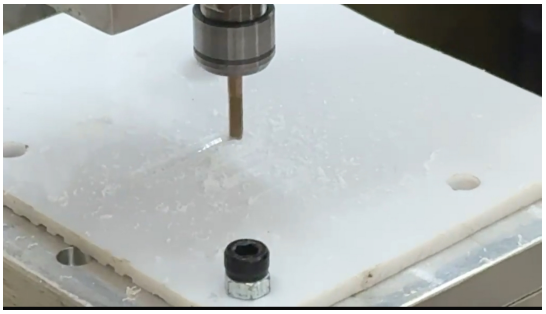

Figure 5: Physical slot cutting underway on the Mitsubishi RV-8CRL arm.

Figure 6: Laser displacement sensor measuring the slot depth profile.

7 Slot Depth Measurement and Chatter Compensation

7.1 Observed Ramp Effect

After the initial slot was milled, its depth profile was measured using a laser displacement sensor traversed along the slot at $x = 0$, 10, 20, and 30 mm. The sensor recorded both the height of the workpiece surface adjacent to the slot and the height of the slot floor at each position.

Despite programming a flat 1 mm deep slot, the measured profile showed a consistent upward ramp from entry to exit. The depth near the start of the slot ($x = 0$ mm) was greater than near the end ($x = 30$ mm), with a total variation of approximately 200 micrometers. This ramp is attributed to structural compliance in the robot arm: cutting forces cause the arm to deflect in a configuration-dependent way as it traverses the slot, with the effective stiffness changing as the joint angles evolve along the path. Additionally, any small residual error in the TCP calibration may have introduced a constant depth offset. Together, these effects produce a position-dependent depth error that must be addressed if a flat, uniform slot floor is required.

7.2 Compensation Strategy

To compensate for the upward ramp, a new toolpath was designed in which the programmed depth decreases gradually along the slot length. Rather than

commanding a constant depth of 1 mm, the toolpath included a slight downward plunge at the entry that reduced progressively toward the exit, so that the deliberately varying depth and the compliance-induced ramp would partially cancel each other, yielding a flatter slot floor overall.

This is a first-order linear compensation. A more rigorous approach would involve mapping the full depth error field across the workpiece and fitting a smooth compensation surface tuned to the robot's compliance at each configuration. The approach used here is an initial proof-of-concept that demonstrates the feasibility of toolpath-based compensation.

7.3 Measured Results

Table 1 presents the laser displacement sensor readings at each measurement position, together with the computed absolute slot depth (the difference between the slot floor reading and the surface reading, reported as a positive value), for both the uncompensated and compensated cases.

Table 1: Laser displacement measurements (mm) along the slot. Surface and Slot are raw sensor readings (negative = below the reference). Depth = |Slot−Surface| gives the absolute cut depth as a positive value.

	No Compensation			**With Compensation**		
x	Surf.	Slot	Depth	Surf.	Slot	Depth
0 mm	−0.078	−1.183	1.105	−0.050	−1.108	1.058
10 mm	−0.149	−1.165	1.016	−0.066	−1.092	1.026
20 mm	−0.141	−1.090	0.949	−0.135	−1.128	0.993
30 mm	−0.212	−1.086	0.874	−0.234	−1.210	0.976

Without compensation, the absolute slot depth ranged from 1.105 mm at $x = 0$ mm down to 0.874 mm at $x = 30$ mm, a total variation of approximately 231 micrometers. With the compensated toolpath, the depth ranged from 1.058 mm to 0.976 mm, a variation of approximately 82 micrometers. This is a reduction of over 60% in depth non-uniformity, demonstrating that even a simple first-order correction applied at the toolpath level can meaningfully improve the dimensional output of a robotic milling operation.

8 Curved Surface Milling: Study and Plans

One of the original goals of the project was to demonstrate the robot's ability to mill a curved surface, not just a flat slot. Curved surfaces are where a robot arm genuinely outperforms a conventional 3-axis CNC machine: the arm can continuously reorient the tool to remain normal to the surface, eliminating the scalloping that would otherwise result. This is precisely the kind of task that makes robotic machining attractive for complex part geometries, and with the workflow established in this project, machining arbitrary curved geometries becomes a realistic proposition.

To that end, a curved surface geometry was modeled in Fusion 360 and a multi-axis toolpath was planned. The strategy explored was a two-stage approach: a **multi-axis roughing pass** to remove the bulk of material, followed by a **multi-axis finishing pass** at a finer step-over to achieve the desired surface quality. Several parameters were studied during this planning phase, including step-over values to trade off between machining time and surface finish, scallop height limits, tool axis tilt relative to the surface normal, and the linking and retract strategy between passes. Different entry moves were also considered to minimise air cutting time and avoid dragging the tool across the surface on re-entry.

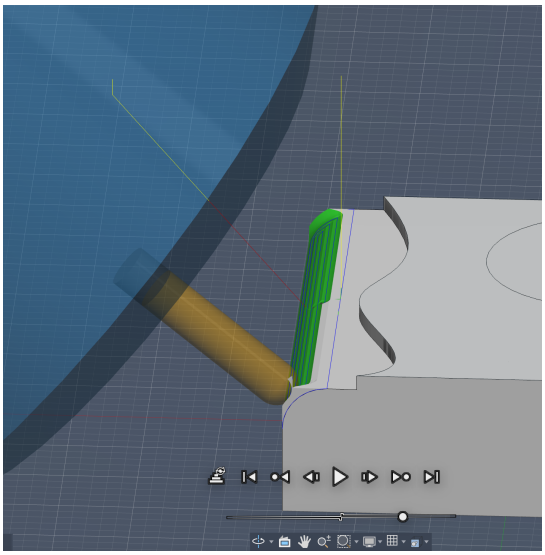

Figure 7: Planned ball-end mill toolpath for curved surface milling in Fusion 360.

The toolpath was also imported into RoboDK and the full robot simulation was run to verify reachability and check for singularities across the curved surface.

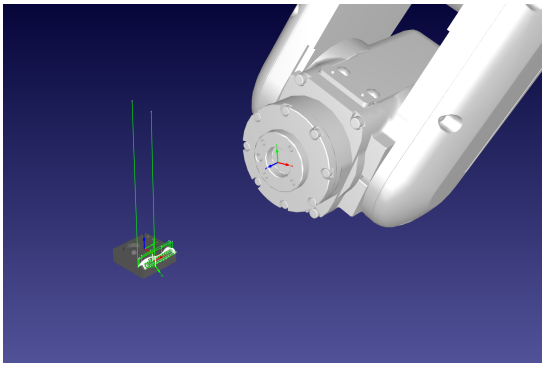

Figure 8: RoboDK simulation of the planned curved surface milling toolpath.

Despite completing the planning and simulation, the physical cutting was not carried out due to the unavailability of a ball-end mill of the correct dimensions. A flat-end mill leaves a stepped profile on curved surfaces and is not suitable for smooth curved surface finishing. A ball-end mill, with its hemispherical cutting tip, is required to properly interpolate the surface. This remains the most immediate next step for this project. Once a suitable ball-end mill is available, the workflow developed here can be applied directly: the Fusion CAM environment, RoboDK simulation pipeline, and RT ToolBox3 execution path are all in place and ready.

9 Companion Video Tutorial

A video tutorial accompanies this report. The video walks through the complete workflow described here, from the CAD setup in Fusion 360 to the physical cutting on the robot, including the TCP calibration procedure, the RoboDK simulation steps, and the RT ToolBox3 program transfer process. Viewers unfamiliar with any of the individual software tools should find the video a useful complement to the written steps in this report.

10 Future Work

10.1 Workflow Optimization

One of the most practical improvements would be to streamline the overall setup process. TCP calibration and frame alignment currently require careful manual work and are a significant source of both time cost and systematic error. Developing a standardized calibration aid, or a dedicated fixture for repeatable TCP measurement, would help considerably. Automating the post-processing of the `.prg` file to strip the RoboDK preamble and populate controller-specific parameters would also reduce setup time on each new run.

10.2 Chatter and Compliance Control

The compliance-induced depth error seen in this project is a fundamental limitation of serial robot arms for milling. Practical improvements could include optimizing joint configurations along the toolpath to keep the arm in stiffer poses, adapting feed rate or spindle speed in real time using force feedback, and building a full compliance map of the robot across its workspace to generate pre-compensated toolpaths. More rigorous compensation, beyond the first-order linear correction used here, could bring depth variation down to within a few tens of micrometers.

10.3 ROS-Based Controller Integration

The CR800D controller supports network communication that can, in principle, be integrated with the Robot Operating System (ROS). A ROS interface would enable real-time joint state monitoring, closed-loop force feedback during cutting, and more flexible program generation. It would also allow the milling setup to be incorporated into larger automation workflows where sensing, planning, and actuation are managed within a unified ROS stack. Future work should explore establishing a ROS driver for the CR800D, building on existing open-source Mitsubishi ROS packages, and using it to implement adaptive control strategies for chatter mitigation.

10.4 Vision-Guided Workpiece Registration

The workpiece frame was defined manually in this project, which is both time-consuming and a source of positioning error. A vision system, such as a structured-light scanner or stereo camera, could automatically detect the workpiece pose before machining and update the reference frame accordingly, eliminating setup errors. In a more advanced form, vision feedback during cutting could be used to detect surface deviations and adapt the toolpath in real time.

10.5 Ball-End Milling of Curved Surfaces

As described in Section 8, the toolpath planning and simulation for curved surface milling have already been completed. The immediate next step is to obtain a ball-end mill of the correct dimensions and execute the planned program. This would validate the full multi-axis workflow and, given the pipeline

already in place, enable machining of practically any surface geometry that can be modeled in Fusion and simulated in RoboDK.

10.6 Full Field Depth Compensation

The linear compensation used in this project relied on only four measurement points. A more rigorous approach would measure depth error on a dense grid across the workpiece, fit a smooth compensation surface to the data, and incorporate it into the toolpath geometry. This would require a systematic measurement protocol and offline data processing, but could substantially improve depth uniformity across more complex milled surfaces.

10.7 Force Sensing and Contact Monitoring

A force/torque sensor at the robot wrist would provide real-time detection of unexpected contact for safety, and data on cutting forces that could be used to tune machining parameters, monitor tool wear, or identify when the robot is operating in a low-stiffness configuration where depth errors are more likely.

11 Conclusion

This project demonstrated a complete robotic milling workflow using the Mitsubishi RV-8CRL, from CAD design in Fusion 360 through RoboDK simulation and RT Toolbox3 execution to physical cutting and measurement. The major challenges encountered included an initial delay in procuring the RoboDK license, which postponed progress by approximately one week. Subsequently, significant time was spent on accurate TCP calibration, which proved critical for achieving reliable tool positioning. Additionally, attempts to directly upload and execute programs from RoboDK on the robot controller faced communication issues over TCP/IP, leading to inconsistent execution. This required a transition to RT Toolbox3 for program deployment and validation, and the development of a more robust and reliable workflow pipeline, which added to the overall project timeline.

The chatter compensation experiment showed that a simple linear correction applied at the toolpath level can reduce depth variation from approximately 200 micrometers to around 85 micrometers, a reduction of over 60%. This suggests that more sophisticated strategies, such as full field compliance mapping and adaptive real-time control, could bring robotic milling accuracy significantly closer to what is achievable with rigid machine tools.

Planning and simulation work for curved surface milling was also completed, with physical execution awaiting a suitable ball-end mill. The broader takeaway is that once this workflow is in place, it can be applied to parts of essentially any geometry that can be modeled in Fusion and simulated in RoboDK, making the setup a genuinely versatile robotic machining platform.